

ЗАДАНИЕ

1. Развернуть виртуальную машину с ОС Linux Ubuntu или Ubuntu Server. Использование иных дистрибутивов допускается, но осуществляется под вашу ответственность.
2. Настроить сетевое подключение в режиме моста (bridge).
3. Зарегистрируйтесь на платформе GitHub (<https://github.com/>) и создайте личный репозиторий.
4. Установите Git на виртуальную машину.
5. Настройте SSH-подключение к GitHub.
6. создайте новый репозиторий на GitHub.
7. Настройте пользователя Git (логин и почта)
8. клонируйте созданный репозиторий к себе на компьютер по ssh.
9. Создайте ветку develop
10. Переключитесь на созданную ветку
11. Реализуйте простое консольное приложение, с выводом произвольной информации. В моем случае Python
12. добавьте в проект файл .github/workflows/ci.yml, скопировав предоставленный шаблон CI-процесса. В моем случае workflow для python.

```
name: CI Check

on:
  push:
    branches: [ "main" ]
  pull_request:
    branches: [ "main" ]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: '3.10'

      - name: Run script
        run: python main.py
```

(В случае использования другого ЯП необходимо переконфигурировать скрипт).

13. Выполните коммит изменений и отправьте их в удалённый репозиторий с помощью `git push`. После отправки кода GitHub Actions автоматически запустит ваш workflow:

- создаст виртуальную среду,
- выполнит указанные в `ci.yml` команды,
- и сообщит о результате выполнения (успех или ошибка).

Часть 2: Расширение CI/CD-процесса: проверка стиля кода и запуск тестов

1. Добавьте в свой репозиторий мини приложение с файлом Шаблон проекта представлен в ссылке.

(<https://drive.google.com/drive/folders/1hC6-oFiS7o73PUx14pUa63l7yBdlv5f9?usp=sharing>)

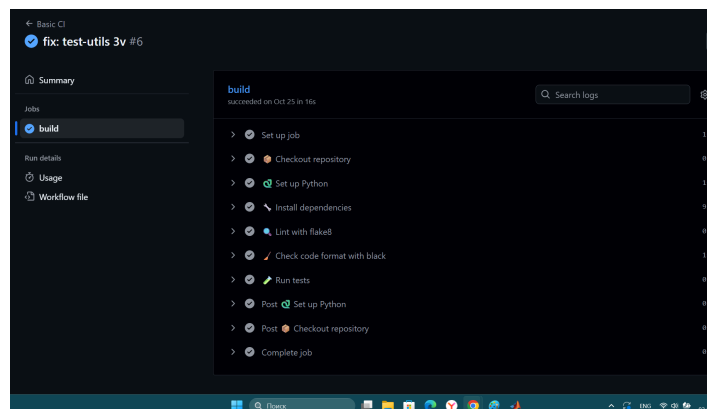
2. Расширьте существующий workflow (файл `.github/workflows/ci.yml`), добавив в него следующие этапы:

- Установка необходимых инструментов: таких как линтер (в случае python например, `flake8` или `ruff`), форматтер (например, `black`) и тестовый фреймворк (например, `pytest` для python).
- Запуск линтера — для проверки соблюдения правил стиля и выявления потенциальных ошибок.
- Запуск форматтера в режиме проверки, чтобы убедиться, что код соответствует установленному форматированию.
- Запуск тестов — с использованием имеющейся мини-программы с тестами (предоставленной ранее) или вашей собственной.

3. Проведите проверку отказоустойчивости CI-процесса, намеренно внося в код следующие изменения:

- нарушьте стиль оформления (например, неправильные отступы, отсутствие пробелов и т.п.).
- Выполните `git commit` и `git push`.
- Убедитесь, что GitHub Actions корректно фиксирует ошибки: этапы линтинга, форматирования или тестирования должны завершиться с ошибкой.
- Устраните все выявленные проблемы: восстановите работоспособность программы и приведите оформление в соответствие с принятыми стандартами.
- Выполните повторный коммит и отправку (`push`).
- Убедитесь, что все шаги CI-процесса проходят успешно.

В случае успешной работы пайплайн будет выглядеть следующим образом.



4. Создайте телеграмм бота

5. Добавьте в workflow отправку сообщения в телеграмм бота о статусе выполнения

- Включите в отчёт:
- Код программы (main.py и тестов),
- Содержимое ci.yml,
- Скриншоты результатов выполнения workflow на GitHub Actions с ошибками (после «поломки» кода) и без ошибок (после исправления). Скриншоты должны чётко отображать статус каждого шага (lint, format check, test).

Доп. задание

Требуется две виртуальные машины (VM) с установленной операционной системой Linux.

Обе виртуальные машины должны находиться в одной сети и быть доступны друг для друга (проверяется командой ping).

Шаги выполнения:

1. Установка Ansible

Установите Ansible на управляющий узел (первую виртуальную машину).

2. Настройка инвентарного файла

Создайте файл hosts, в котором укажите IP-адрес управляемого узла (второй виртуальной машины). Убедитесь, что Ansible может подключиться к нему по SSH (настройте ключи при необходимости).

3. Создание playbook'a

Напишите файл playbook.yml, в котором последовательно опишите следующие задачи для управляемого узла:

- Обновление списка пакетов и обновление всех установленных пакетов (apt update && apt upgrade или аналог для используемого дистрибутива).
- Создание нового пользователя ansible_{Ваша фамилия}
- Установка системы контроля версий Git.
- Установка СУБД PostgreSQL.
- Установка веб-сервера Nginx.
- Создание небольшого index.html и передача его в определенную директорию для Nginx
- Запуск nginx

Шаблон:

- name: Настройка сервера через Ansible

hosts: {файл hosts}

become: true

tasks:

- name: Обновить список пакетов

apt:

update_cache: true

cache_valid_time: 3600

ЗАДАНИЕ: Добавьте задачу для установки Git

Используйте модуль apt и укажите имя пакета

ЗАДАНИЕ : Создайте нового пользователя ansible_{Ваша фамилия}

ЗАДАНИЕ: Установите PostgreSQL

ЗАДАНИЕ: Установите nginx

ЗАДАНИЕ: Создайте небольшой файл index.html

ЗАДАНИЕ: Запустите nginx

4. Запуск playbook'a

Выполните playbook с помощью определенной команды. Убедитесь, что все задачи завершились успешно (без ошибок).

5. Проверка результата

Подключитесь к управляемому узлу (второй ВМ) и убедитесь, что: Git, PostgreSQL и Nginx установлены и работают корректно.

Подготовить отчет, содержащий скриншоты и небольшое описание всех ключевых этапов выполнения.

Загрузить отчет на портал LXP в КТ - 4 (при необходимости можно продублировать файлом в Telegram).

